

Anaconda

Your Toolbox for Python Project

Anaconda Cloud

Create a New Project

Create a New Notebook

My Projects

Code Snippets

Manage Code Snippets

Anaconda AI Assistant

Chat-based Python Help

```
[41]: # Load Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Lasso
from sklearn.linear_model import ElasticNet
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.ensemble import AdaBoostRegressor
from sklearn.metrics import mean_squared_error
from pandas import read_csv
```

```
[42]: filename = r"C:\Users\hp\Downloads\WineQT.csv"
dataset = read_csv(filename)
```

```
[43]: # shape
print(dataset.shape)

(1143, 13)
```

```
[44]: # types
print(dataset.dtypes)

fixed acidity      float64
volatile acidity   float64
citric acid        float64
residual sugar     float64
chlorides          float64
free sulfur dioxide float64
total sulfur dioxide float64
density            float64
pH                float64
sulphates          float64
alcohol            float64
quality            int64
Id                int64
dtype: object
```

```
[48]: #head
print(dataset.head(5))

   fixed acidity  volatile acidity  citric acid  residual sugar  chlorides \
0         7.4             0.70         0.00           1.9        0.076
1         7.8             0.88         0.00           2.6        0.098
2         7.8             0.76         0.04           2.3        0.092
3        11.2             0.28         0.56           1.9        0.075
4         7.4             0.70         0.00           1.9        0.076

   free sulfur dioxide  total sulfur dioxide  density  pH  sulphates \
0             11.0             34.0  0.9978  3.51         0.56
1             25.0             67.0  0.9968  3.20         0.68
2             15.0             54.0  0.9970  3.26         0.65
3             17.0             60.0  0.9980  3.16         0.58
4             11.0             34.0  0.9978  3.51         0.56

   alcohol  quality  Id
0       9.4         5  0
1       9.8         5  1
2       9.8         5  2
3       9.8         6  3
4       9.4         5  4
```

```
[49]: #descriptions
pd.set_option('display.precision',1)
print(dataset.describe())

   fixed acidity  volatile acidity  citric acid  residual sugar \
count      1143.0             1143.0      1.1e+03      1143.0
mean         8.3              0.5      2.7e-01          2.5
std          1.7              0.2      2.0e-01          1.4
min          4.6              0.1      0.0e+00          0.9
25%          7.1              0.4      9.0e-02          1.9
50%          7.9              0.5      2.5e-01          2.2
75%          9.1              0.6      4.2e-01          2.6
max         15.9             1.6      1.0e+00          15.5

   chlorides  free sulfur dioxide  total sulfur dioxide  density  pH \
count      1.1e+03             1143.0             1143.0  1.1e+03  1143.0
mean      8.7e-02              15.6              45.9  1.0e+00   3.3
std       4.7e-02              10.3              32.8  1.9e-03   0.2
min       1.2e-02               1.0               6.0  9.9e-01   2.7
25%       7.0e-02              7.0              21.0  1.0e+00   3.2
50%       7.9e-02             13.0              37.0  1.0e+00   3.3
75%       9.0e-02             21.0              61.0  1.0e+00   3.4
```

max	6.1e-01	68.0	289.0	1.0e+00	4.0
-----	---------	------	-------	---------	-----

	sulphates	alcohol	quality	Id
count	1143.0	1143.0	1143.0	1143.0
mean	0.7	10.4	5.7	805.0
std	0.2	1.1	0.8	464.0
min	0.3	8.4	3.0	0.0
25%	0.6	9.5	5.0	411.0
50%	0.6	10.2	6.0	794.0
75%	0.7	11.1	6.0	1209.5
max	2.0	14.9	8.0	1597.0

```
[55]: #correlation
pd.set_option('display.precision',2)
print(dataset.corr(method='pearson'))
```

	fixed acidity	volatile acidity	citric acid	\
fixed acidity	1.00	-2.51e-01	0.67	
volatile acidity	-0.25	1.00e+00	-0.54	
citric acid	0.67	-5.44e-01	1.00	
residual sugar	0.17	-5.75e-03	0.18	
chlorides	0.11	5.63e-02	0.25	
free sulfur dioxide	-0.16	-1.96e-03	-0.06	
total sulfur dioxide	-0.11	7.77e-02	0.04	
density	0.68	1.65e-02	0.38	
pH	-0.69	2.21e-01	-0.55	
sulphates	0.17	-2.76e-01	0.33	
alcohol	-0.08	-2.04e-01	0.11	
quality	0.12	-4.07e-01	0.24	

	residual sugar	chlorides	free sulfur dioxide	\
fixed acidity	1.72e-01	0.11	-1.65e-01	
volatile acidity	-5.75e-03	0.06	-1.96e-03	
citric acid	1.76e-01	0.25	-5.76e-02	
residual sugar	1.00e+00	0.07	1.65e-01	
chlorides	7.09e-02	1.00	1.53e-02	
free sulfur dioxide	1.65e-01	0.02	1.00e+00	
total sulfur dioxide	1.91e-01	0.05	6.61e-01	
density	3.80e-01	0.21	-5.42e-02	
pH	-1.17e-01	-0.28	7.28e-02	
sulphates	1.75e-02	0.37	3.44e-02	
alcohol	5.84e-02	-0.23	-4.71e-02	
quality	2.20e-02	-0.12	-6.33e-02	

	total sulfur dioxide	density	pH	sulphates	alcohol	\
fixed acidity	-0.11	0.68	-0.69	0.17	-0.08	
volatile acidity	0.08	0.02	0.22	-0.28	-0.20	
citric acid	0.04	0.38	-0.55	0.33	0.11	
residual sugar	0.19	0.38	-0.12	0.02	0.06	
chlorides	0.05	0.21	-0.28	0.37	-0.23	
free sulfur dioxide	0.66	-0.05	0.07	0.03	-0.05	
total sulfur dioxide	1.00	0.05	-0.06	0.03	-0.19	
density	0.05	1.00	-0.35	0.14	-0.49	
pH	-0.06	-0.35	1.00	-0.19	0.23	
sulphates	0.03	0.14	-0.19	1.00	0.09	
alcohol	-0.19	-0.49	0.23	0.09	1.00	
quality	-0.18	-0.18	-0.05	0.26	0.48	

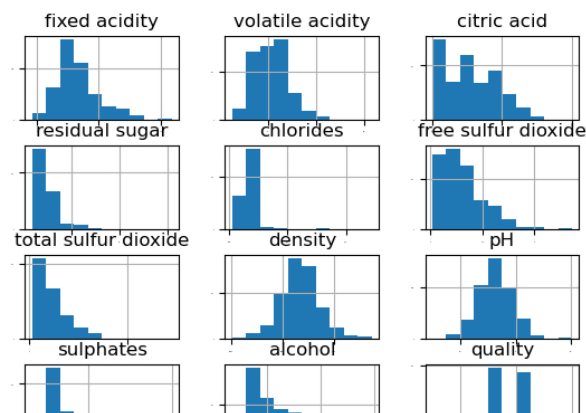
	quality
fixed acidity	0.12
volatile acidity	-0.41
citric acid	0.24
residual sugar	0.02
chlorides	-0.12
free sulfur dioxide	-0.06
total sulfur dioxide	-0.18
density	-0.18
pH	-0.05
sulphates	0.26
alcohol	0.48
quality	1.00

```
[53]: if 'Id' in dataset.columns:
dataset = dataset.drop('Id', axis=1)
```

```
[54]: # shape
print(dataset.shape)

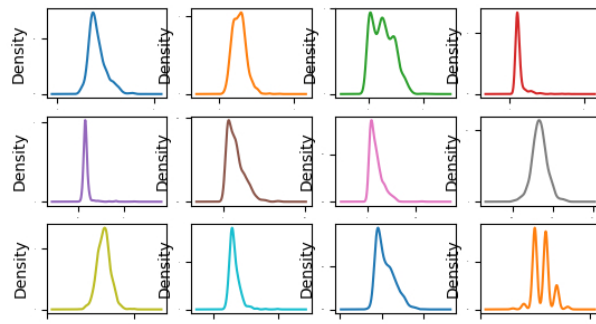
(1143, 12)
```

```
[56]: # histograms
dataset.hist(sharex=False, sharey=False, xlabelsize=1, ylabelsize=1)
pyplot.show()
```

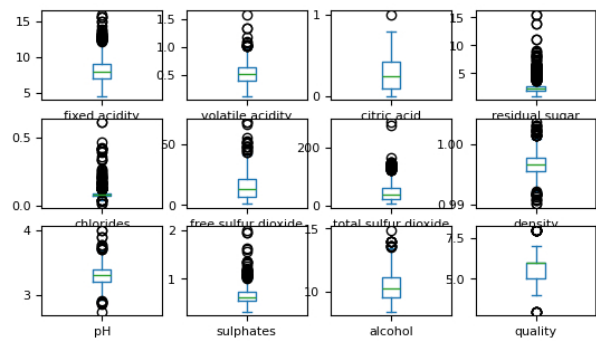




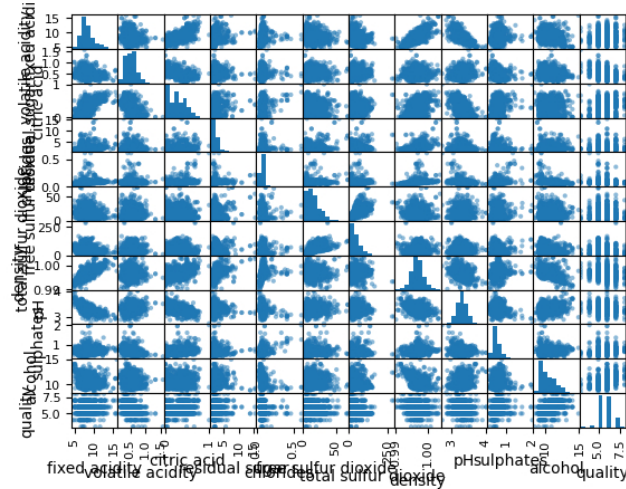
```
[57]: # density
dataset.plot(kind='density', subplots=True, layout=(4,4), sharex=False, legend=False,
          fontsize=1)
pyplot.show()
```



```
[58]: # box and whisker plots
dataset.plot(kind='box', subplots=True, layout=(4,4), sharex=False, sharey=False,
          fontsize=8)
pyplot.show()
```



```
[60]: # scatter plot matrix
from pandas.plotting import scatter_matrix
scatter_matrix(dataset)
pyplot.show()
```

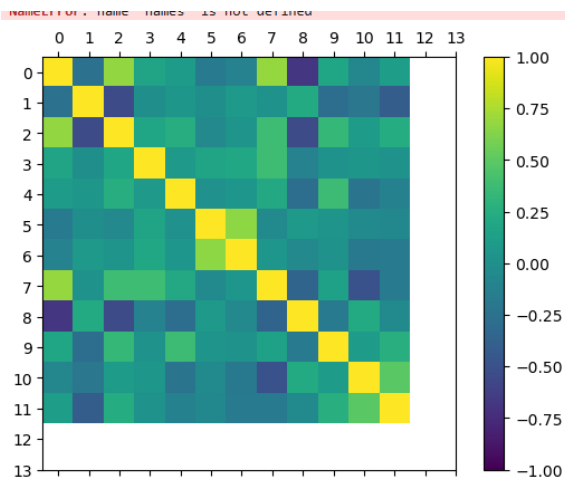


```
[63]: # correlation matrix
fig = pyplot.figure()
ax = fig.add_subplot(111)
cax = ax.matshow(dataset.corr(), vmin=-1, vmax=1, interpolation='none')
fig.colorbar(cax)
ticks = numpy.arange(0,14,1)
ax.set_xticks(ticks)
ax.set_yticks(ticks)
ax.set_xticklabels(names)
ax.set_yticklabels(names)
pyplot.show()
```

NameError Traceback (most recent call last)

```
Cell In[63], line 9
      7 ax.set_xticks(ticks)
      8 ax.set_yticks(ticks)
---->  9 ax.set_xticklabels(names)
      10 ax.set_yticklabels(names)
      11 pyplot.show()

NameError: name 'names' is not defined
```



```
[65]: # Split-out validation dataset
array = dataset.values
X = array[:,0:11]
Y = array[:,11]
validation_size = 0.20
seed = 7
X_train, X_validation, Y_train, Y_validation = train_test_split(X, Y,
test_size=validation_size, random_state=seed)

[66]: # Test options and evaluation metric
num_folds = 10
seed = 7
scoring = 'neg_mean_squared_error'

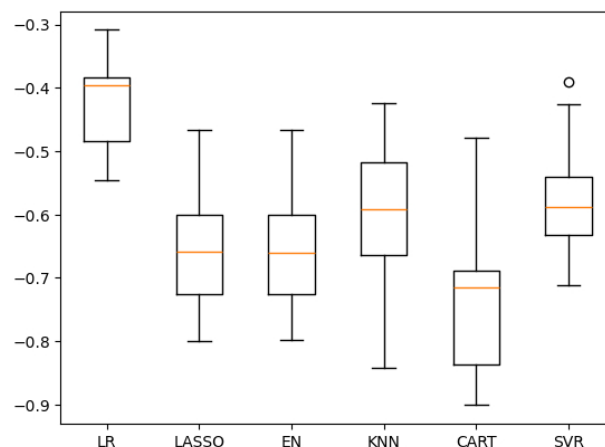
[67]: # Spot-Check Algorithms
models = []
models.append(('LR', LinearRegression()))
models.append(('LASSO', Lasso()))
models.append(('EN', ElasticNet()))
models.append(('KNN', KNeighborsRegressor()))
models.append(('CART', DecisionTreeRegressor()))
models.append(('SVR', SVR()))

[75]: # evaluate each model in turn
results = []
names = []
for name, model in models:
    kfold = KFold(n_splits=num_folds,shuffle=True, random_state=seed)
    cv_results = cross_val_score(model, X_train, Y_train, cv=kfold, scoring=scoring)
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
    print(msg)

LR: -0.424679 (0.074539)
LASSO: -0.649565 (0.104702)
EN: -0.648923 (0.105121)
KNN: -0.593168 (0.120578)
CART: -0.727759 (0.127617)
SVR: -0.570598 (0.095957)

[76]: # Compare Algorithms
fig = pyplot.figure()
fig.suptitle('Algorithm Comparison')
ax = fig.add_subplot(111)
pyplot.boxplot(results)
ax.set_xticklabels(names)
pyplot.show()
```

Algorithm Comparison

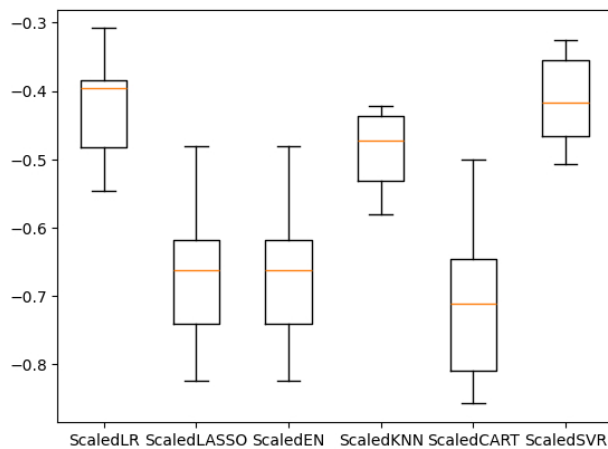


```
[78]: # Standardize the dataset
pipelines = []
pipelines.append(('ScaledLR', Pipeline([('Scaler', StandardScaler()), ('LR',
LinearRegression())])))
pipelines.append(('ScaledLASSO', Pipeline([('Scaler', StandardScaler()), ('LASSO',
Lasso())])))
pipelines.append(('ScaledEN', Pipeline([('Scaler', StandardScaler()), ('EN',
ElasticNet())])))
pipelines.append(('ScaledKNN', Pipeline([('Scaler', StandardScaler()), ('KNN',
KNeighborsRegressor())])))
pipelines.append(('ScaledCART', Pipeline([('Scaler', StandardScaler()), ('CART',
DecisionTreeRegressor())])))
pipelines.append(('ScaledSVR', Pipeline([('Scaler', StandardScaler()), ('SVR', SVR())])))
results = []
names = []
for name, model in pipelines:
    kfold = KFold(n_splits=num_folds, shuffle=True, random_state=seed)
    cv_results = cross_val_score(model, X_train, Y_train, cv=kfold, scoring=scoring)
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
    print(msg)

ScaledLR: -0.424679 (0.074539)
ScaledLASSO: -0.667636 (0.102382)
ScaledEN: -0.667636 (0.102382)
ScaledKNN: -0.486283 (0.057651)
ScaledCART: -0.716842 (0.106840)
ScaledSVR: -0.413750 (0.061243)
```

```
[79]: # Compare Algorithms
fig = pyplot.figure()
fig.suptitle('Scaled Algorithm Comparison')
ax = fig.add_subplot(111)
pyplot.boxplot(results)
ax.set_xticklabels(names)
pyplot.show()
```

Scaled Algorithm Comparison



```
[85]: from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import KFold, GridSearchCV

# Scale
scaler = StandardScaler().fit(X_train)
rescaledX = scaler.transform(X_train)

neighbors = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21]
param_grid = dict(n_neighbors=neighbors)

model = KNeighborsClassifier()

kfold = KFold(n_splits=num_folds, shuffle=True, random_state=seed)

grid = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    scoring=scoring,
    cv=kfold
)

grid_result = grid.fit(rescaledX, Y_train)

print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))

means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']

for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))

Best: -0.550490 using {'n_neighbors': 21}
-0.679419 (0.131297) with: {'n_neighbors': 1}
-0.793156 (0.086859) with: {'n_neighbors': 3}
```

```
-0.647707 (0.092227) with: {'n_neighbors': 5}
-0.626935 (0.072127) with: {'n_neighbors': 7}
-0.588641 (0.073941) with: {'n_neighbors': 9}
-0.572324 (0.081846) with: {'n_neighbors': 11}
-0.583290 (0.086041) with: {'n_neighbors': 13}
-0.563509 (0.076403) with: {'n_neighbors': 15}
-0.561312 (0.074081) with: {'n_neighbors': 17}
-0.552652 (0.094659) with: {'n_neighbors': 19}
-0.550490 (0.090243) with: {'n_neighbors': 21}
```

```
[86]: # Tune scaled SVM
scaler = StandardScaler().fit(X_train)
rescaledX = scaler.transform(X_train)
c_values = [0.1, 0.3, 0.5, 0.7, 0.9, 1.0, 1.3, 1.5, 1.7, 2.0]
kernel_values = ['linear', 'poly', 'rbf', 'sigmoid']
param_grid = dict(C=c_values, kernel=kernel_values)
model = SVR()
kfold = KFold(n_splits=num_folds)
grid = GridSearchCV(estimator=model, param_grid=param_grid, scoring=scoring, cv=kfold)
grid_result = grid.fit(rescaledX, Y_train)
print("best score:", grid_result.best_score_, grid_result.best_params_)
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print(mean, stdev, param)
```

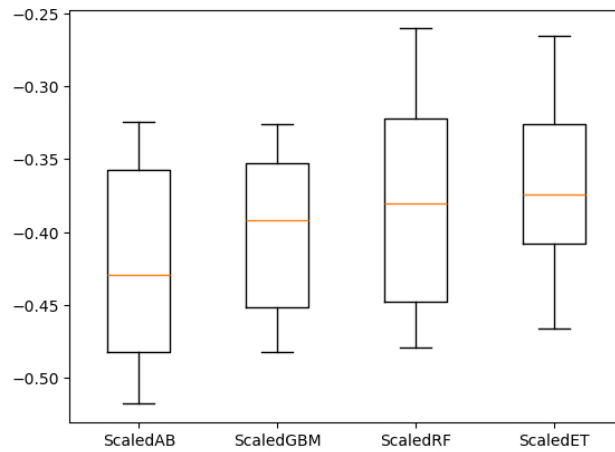
```
best score: -0.41084191275068704 {'C': 0.7, 'kernel': 'rbf'}
-0.42791002077858453 0.0772617590290935 {'C': 0.1, 'kernel': 'linear'}
-0.6047043302718046 0.280927727360654 {'C': 0.1, 'kernel': 'poly'}
-0.43001418479419395 0.09306051062311631 {'C': 0.1, 'kernel': 'rbf'}
-0.6089502190460605 0.10844785463807761 {'C': 0.1, 'kernel': 'sigmoid'}
-0.42847377107975076 0.07726588545851436 {'C': 0.3, 'kernel': 'linear'}
-0.6362398582530692 0.28083414865115214 {'C': 0.3, 'kernel': 'poly'}
-0.4147636917283125 0.08739371148684917 {'C': 0.3, 'kernel': 'rbf'}
-5.660778146584333 0.8850885904147057 {'C': 0.3, 'kernel': 'sigmoid'}
-0.4284651227696418 0.0773090186430258 {'C': 0.5, 'kernel': 'linear'}
-0.6912729421980401 0.29004668216581225 {'C': 0.5, 'kernel': 'poly'}
-0.4109744000296908 0.08838937938655314 {'C': 0.5, 'kernel': 'rbf'}
-15.099414057987275 3.3004912681753207 {'C': 0.5, 'kernel': 'sigmoid'}
-0.4285469332547539 0.07716077255674894 {'C': 0.7, 'kernel': 'linear'}
-0.7589635584359143 0.4153384957985282 {'C': 0.7, 'kernel': 'poly'}
-0.41084191275068704 0.08899248643868843 {'C': 0.7, 'kernel': 'rbf'}
-32.00913441384042 6.4272051104374555 {'C': 0.7, 'kernel': 'sigmoid'}
-0.4285205450884824 0.07726332908238197 {'C': 0.9, 'kernel': 'linear'}
-0.7999408297046615 0.5333320525714943 {'C': 0.9, 'kernel': 'poly'}
-0.41360799547934946 0.0894756144279002 {'C': 0.9, 'kernel': 'rbf'}
-50.37124814589495 13.653766316169675 {'C': 0.9, 'kernel': 'sigmoid'}
-0.4285239137922268 0.0773277153412326 {'C': 1.0, 'kernel': 'linear'}
-0.8262421293310462 0.6010029674201575 {'C': 1.0, 'kernel': 'poly'}
-0.41504458978281783 0.08960074726148824 {'C': 1.0, 'kernel': 'rbf'}
-58.48541319757071 16.013780726189697 {'C': 1.0, 'kernel': 'sigmoid'}
-0.42860857438445266 0.07740785207883973 {'C': 1.3, 'kernel': 'linear'}
-0.908891115563537 0.7587294822427725 {'C': 1.3, 'kernel': 'poly'}
-0.4197678469412698 0.08955981029093524 {'C': 1.3, 'kernel': 'rbf'}
-98.08311960298178 27.21714932460939 {'C': 1.3, 'kernel': 'sigmoid'}
-0.42864468423922786 0.07738338547161858 {'C': 1.5, 'kernel': 'linear'}
-0.9465663120010805 0.8235960835156585 {'C': 1.5, 'kernel': 'poly'}
-0.42206325125720107 0.08911655420741743 {'C': 1.5, 'kernel': 'rbf'}
-132.6309367371147 40.52748880318693 {'C': 1.5, 'kernel': 'sigmoid'}
-0.4285390981041267 0.07735686279063697 {'C': 1.7, 'kernel': 'linear'}
-0.9783828112423943 0.8747319266406349 {'C': 1.7, 'kernel': 'poly'}
-0.42404242504907136 0.08833272708889044 {'C': 1.7, 'kernel': 'rbf'}
-168.97153058026674 48.4196115272221 {'C': 1.7, 'kernel': 'sigmoid'}
-0.42858806265743954 0.0773271178372109 {'C': 2.0, 'kernel': 'linear'}
-1.0334545286018058 0.9654070082821817 {'C': 2.0, 'kernel': 'poly'}
-0.42680043815203356 0.08882490268431081 {'C': 2.0, 'kernel': 'rbf'}
-246.69944511865623 67.23940050701187 {'C': 2.0, 'kernel': 'sigmoid'}
```

```
[87]: # ensembles
ensembles = []
ensembles.append(('ScaledAB', Pipeline([('Scaler', StandardScaler()), ('AB',
AdaBoostRegressor())])))
ensembles.append(('ScaledGBM', Pipeline([('Scaler', StandardScaler()), ('GBM',
GradientBoostingRegressor())])))
ensembles.append(('ScaledRF', Pipeline([('Scaler', StandardScaler()), ('RF',
RandomForestRegressor())])))
ensembles.append(('ScaledET', Pipeline([('Scaler', StandardScaler()), ('ET',
ExtraTreesRegressor())])))
results = []
names = []
for name, model in ensembles:
    kfold = KFold(n_splits=num_folds, shuffle=True, random_state=seed)
    cv_results = cross_val_score(model, X_train, Y_train, cv=kfold, scoring=scoring)
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
    print(msg)
```

```
ScaledAB: -0.421490 (0.067119)
ScaledGBM: -0.399829 (0.056786)
ScaledRF: -0.380410 (0.074373)
ScaledET: -0.366779 (0.059461)
```

```
[88]: # Compare Algorithms
fig = pyplot.figure()
fig.suptitle('Scaled Ensemble Algorithm Comparison')
ax = fig.add_subplot(111)
pyplot.boxplot(results)
ax.set_xticklabels(names)
pyplot.show()
```

Scaled Ensemble Algorithm Comparison



```
[96]: scaler = StandardScaler().fit(X_train)
rescaledX = scaler.transform(X_train)
param_grid = dict(n_estimators=np.array([50,100,150,200,250,300,350,400]))
model = ExtraTreesRegressor(random_state=seed)
kfold = KFold(n_splits=num_folds, shuffle=True, random_state=seed)
grid = GridSearchCV(estimator=model, param_grid=param_grid, scoring=scoring, cv=kfold)
grid_result = grid.fit(rescaledX, Y_train)
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

```
Best: -0.366525 using {'n_estimators': np.int64(200)}
-0.372453 (0.062490) with: {'n_estimators': np.int64(50)}
-0.369454 (0.065335) with: {'n_estimators': np.int64(100)}
-0.369089 (0.062001) with: {'n_estimators': np.int64(150)}
-0.366525 (0.060130) with: {'n_estimators': np.int64(200)}
-0.367606 (0.063008) with: {'n_estimators': np.int64(250)}
-0.368304 (0.062031) with: {'n_estimators': np.int64(300)}
-0.367079 (0.061244) with: {'n_estimators': np.int64(350)}
-0.367298 (0.061406) with: {'n_estimators': np.int64(400)}
```

```
[90]: # Tune scaled GBM
scaler = StandardScaler().fit(X_train)
rescaledX = scaler.transform(X_train)
param_grid = dict(n_estimators=np.array([50,100,150,200,250,300,350,400]))
model = GradientBoostingRegressor(random_state=seed)
kfold = KFold(n_splits=num_folds, shuffle=True, random_state=seed)
grid = GridSearchCV(estimator=model, param_grid=param_grid, scoring=scoring, cv=kfold)
grid_result = grid.fit(rescaledX, Y_train)
```

```
[91]: print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

```
Best: -0.398989 using {'n_estimators': np.int64(50)}
-0.398989 (0.057601) with: {'n_estimators': np.int64(50)}
-0.405324 (0.059984) with: {'n_estimators': np.int64(100)}
-0.407520 (0.064605) with: {'n_estimators': np.int64(150)}
-0.410782 (0.069676) with: {'n_estimators': np.int64(200)}
-0.416078 (0.072801) with: {'n_estimators': np.int64(250)}
-0.418769 (0.074046) with: {'n_estimators': np.int64(300)}
-0.423092 (0.076838) with: {'n_estimators': np.int64(350)}
-0.427781 (0.079146) with: {'n_estimators': np.int64(400)}
```

```
[94]: # prepare the model
scaler = StandardScaler().fit(X_train)
rescaledX = scaler.transform(X_train)
model = GradientBoostingRegressor(random_state=seed, n_estimators=400)
model.fit(rescaledX, Y_train)
```

```
[94]: GradientBoostingRegressor
GradientBoostingRegressor(n_estimators=400, random_state=7)
```

```
[95]: # prepare the model
scaler = StandardScaler().fit(X_train)
rescaledX = scaler.transform(X_train)
model = GradientBoostingRegressor(random_state=seed, n_estimators=400)
model.fit(rescaledX, Y_train)
```

```
0.3630539797067448
```

```
[ ]:
```

